

DATABASE MANAGEMENT SYSTEM

Vehicle Service Centre Management System



Submitted by:

Reddy Keerthana – 25WU0101107

Morla Karthik – 25WU0102166

Shwechith Charan – 25WU0102261

Course: B. Tech, AIML

Faculty: Dr. Kiran Mayee Adavala

Academic Year: 2026–2027

Abstract

The Vehicle Service Centre Management System is a relational database project designed to organize and manage the major activities of a vehicle service centre. The system maintains customer, vehicle, service booking, job card, mechanic, task, spare-part, inspection, invoice and payment information in a structured database.

The system addresses difficulties associated with manual records, such as duplicated information, difficult service-history tracking, mechanic assignment issues, spare-part tracking problems and billing errors. It provides a centralized database for storing and retrieving service-related information.

The database is designed using relational database principles and normalized up to Third Normal Form (3NF). SQL commands are used for table creation, insertion, updating, deletion, joins, aggregation, nested queries and report generation. A simple Java or Python front end can be connected to MySQL to support CRUD operations and service-centre workflows.

Introduction and Problem Statement

Introduction

Vehicle service centres handle many records every day. These include customer details, vehicle information, service appointments, job cards, repair tasks, mechanic assignments, spare parts, inspections, invoices and payments. When these records are maintained manually, finding information and maintaining consistent service history can become difficult.

The proposed system provides a centralized database to store and manage service-centre information. It allows authorized users to register customers and vehicles, create service bookings, process job cards, assign mechanics, record spare-part usage, generate invoices and maintain service history.

Problem Statement

- Manual records can be lost, duplicated or difficult to update.
- Vehicle service history may be difficult to retrieve quickly.
- Mechanic assignments and job progress may be difficult to track.
- Spare-part usage and stock information may not be maintained accurately.
- Manual invoice calculation can result in errors.
- Generating service-centre reports can be time-consuming.

Therefore, a database-driven system is required to provide organized storage, faster retrieval, better data consistency and efficient management of vehicle service operations.

Objectives and Scope

Objectives

1. Maintain customer and vehicle information.
2. Manage vehicle service bookings.
3. Create and process job cards.
4. Assign mechanics to service tasks.
5. Maintain spare-part information and part usage.
6. Record inspections and repair tasks.
7. Generate invoices and maintain payment records.
8. Track vehicle service history.
9. Generate useful reports using SQL queries.
10. Reduce manual work and improve data consistency.

Scope

The system covers the main operational workflow of a vehicle service centre:

**Customer → Vehicle → Booking → Job Card → Mechanic → Task/Parts → Inspection →
Invoice → Payment → Service History**

The system is intended for service-centre staff who need a structured way to create, update, search and report service-related records.

Software and Hardware Requirements

Software Requirements:

Requirement	Technology / Tool
Operating System	Windows / Linux
Database	MySQL
Front End	Java or Python
Database Connectivity	JDBC or MySQL Connector
Development Environment	VS Code / IntelliJ / Eclipse
Database Tool	MySQL Workbench
Documentation	Microsoft Word
Version Control	GitHub

Hardware Requirements:

Component	Minimum Requirement
Processor	Intel i3 or equivalent
RAM	4 GB or above
Storage	10 GB available
Display	Standard monitor
Input	Keyboard and mouse

6. ER Diagram

The major entities in the system are:

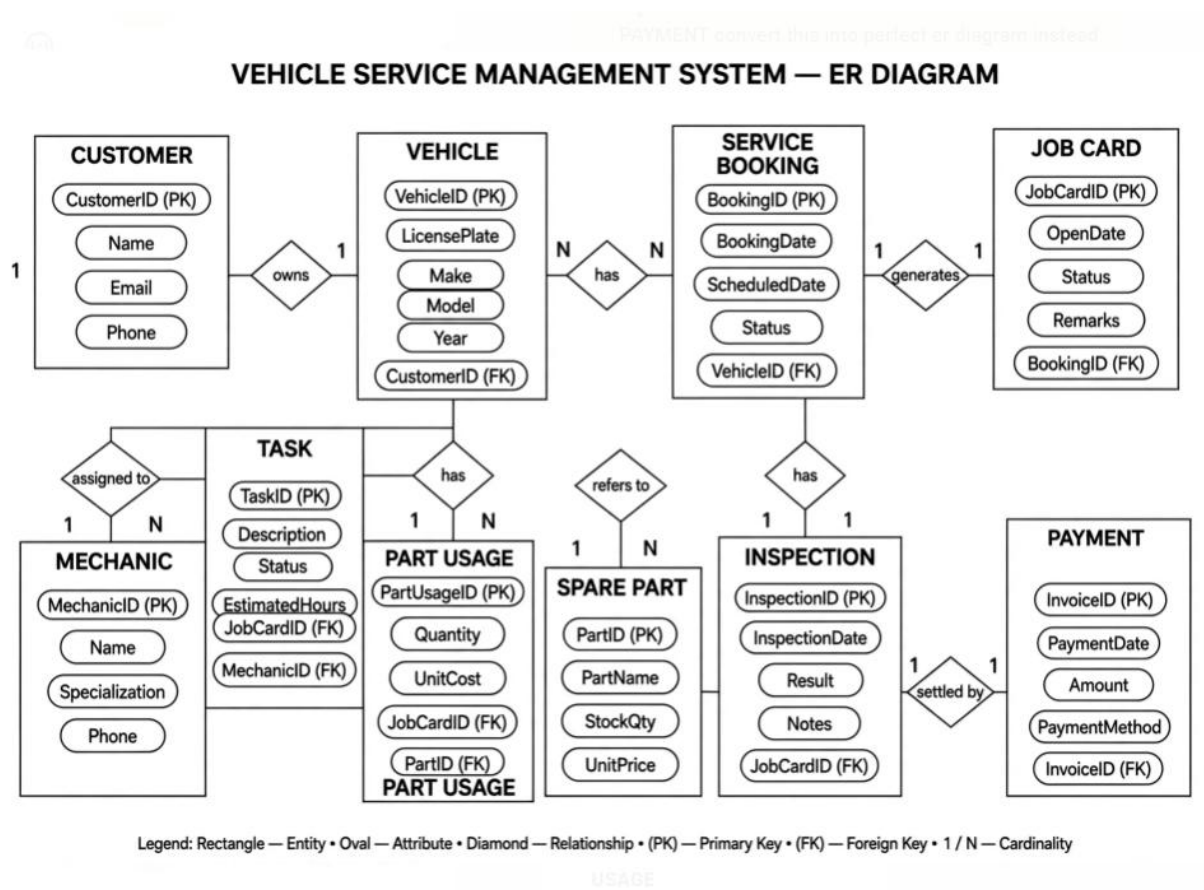
- Customer
- Vehicle
- Service Booking
- Job Card
- Task
- Mechanic
- Assignment
- Spare Part
- Part Usage
- Inspection
- Invoice
- Payment

Main Relationships

- One Customer can own multiple Vehicles.
- One Vehicle can have multiple Service Bookings.
- A Service Booking can generate a Job Card.
- A Job Card can contain multiple Tasks.
- A Mechanic can be assigned to service tasks.
- A Job Card can use multiple Spare Parts.
- An Invoice is generated for a completed service.
- An Invoice can have payment information.
- A Vehicle can have multiple inspection and service-history records.

The ER model represents the major entities and their relationships. A customer can own multiple vehicles. A vehicle can have multiple service bookings. A booking is processed through a job card, which contains tasks and can involve mechanics, spare parts and inspections. Completed work is billed through an invoice, and payment information is recorded against the invoice.

Simple ER Structure:



Relational Schema and Normalization

Relational Schema

Table	Attributes	Primary Key
Customer	Customer_ID, Name, Phone, Email, Address	Customer_ID
Vehicle	Vehicle_ID, Customer_ID, Registration_No, Model, Brand, Vehicle_Type	Vehicle_ID
Service_Booking	Booking_ID, Vehicle_ID, Booking_Date, Service_Date, Service_Type, Status	Booking_ID
Job_Card	Job_ID, Booking_ID, Start_Date, Completion_Date, Status, Remarks	Job_ID
Mechanic	Mechanic_ID, Name, Phone, Specialization	Mechanic_ID
Task	Task_ID, Job_ID, Task_Name, Description, Status	Task_ID
Assignment	Assignment_ID, Task_ID, Mechanic_ID, Assigned_Date	Assignment_ID
Spare_Part	Part_ID, Part_Name, Quantity, Unit_Price	Part_ID
Part_Usage	Usage_ID, Job_ID, Part_ID, Quantity_Used	Usage_ID
Inspection	Inspection_ID, Job_ID, Inspection_Date, Findings, Remarks	Inspection_ID
Invoice	Invoice_ID, Job_ID, Invoice_Date, Total_Amount	Invoice_ID
Payment	Payment_ID, Invoice_ID, Payment_Date, Amount, Payment_Mode	Payment_ID

Normalization:

The database is designed to be normalized up to Third Normal Form (3NF).

First Normal Form (1NF): Each attribute contains atomic values and there are no repeating groups.

Second Normal Form (2NF): Non-key attributes are fully dependent on the complete primary key of their relation.

Third Normal Form (3NF): Non-key attributes do not depend on other non-key attributes. Each relation represents a specific entity or relationship, reducing redundancy and update anomalies.

8. Data Dictionary

Table	Attribute	Data Type	Constraint / Purpose
Customer	Customer_ID	INT	Primary Key
Customer	Name	VARCHAR(100)	NOT NULL
Customer	Phone	VARCHAR(15)	Customer contact
Customer	Email	VARCHAR(100)	Email address
Vehicle	Vehicle_ID	INT	Primary Key
Vehicle	Customer_ID	INT	Foreign Key
Vehicle	Registration_No	VARCHAR(20)	UNIQUE
Service_Booking	Booking_ID	INT	Primary Key
Service_Booking	Vehicle_ID	INT	Foreign Key
Service_Booking	Service_Date	DATE	Service date
Job_Card	Job_ID	INT	Primary Key
Job_Card	Booking_ID	INT	Foreign Key
Mechanic	Mechanic_ID	INT	Primary Key
Spare_Part	Part_ID	INT	Primary Key
Invoice	Invoice_ID	INT	Primary Key
Payment	Payment_ID	INT	Primary Key

The data dictionary defines the main attributes used in the relational database, their data types and the constraints applied to maintain data integrity.

9. SQL Commands Used (DDL and DML) with Sample Outputs

9.1 Table Creation

```
CREATE TABLE Customer (  
    Customer_ID INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Phone VARCHAR(15),  
    Email VARCHAR(100),  
    Address VARCHAR(200)  
);
```

9.2 Insert Record

```
INSERT INTO Customer  
VALUES (1, 'Rahul Kumar', '9876543210',  
        'rahul@gmail.com', 'Hyderabad');
```

9.3 Select Records

```
SELECT * FROM Customer;
```

Customer_ID	Name	Phone	Email
1	Rahul Kumar	9876543210	rahul@gmail.com
2	Priya Sharma	9123456780	priya@gmail.com

9.4 Update Record

```
UPDATE Customer  
SET Phone = '9876501234'  
WHERE Customer_ID = 1;
```

9.5 Delete Record

```
DELETE FROM Customer  
WHERE Customer_ID = 1;
```

9.6 Join Query

```
SELECT c.Name, v.Registration_No, v.Model  
FROM Customer c  
JOIN Vehicle v  
ON c.Customer_ID = v.Customer_ID;
```

10. Queries with Outputs

10.1 Pending Service Jobs

```
SELECT *
FROM Job_Card
WHERE Status = 'Pending';
```

Job_ID	Booking_ID	Status	Remarks
J002	B002	Pending	Waiting for parts
J005	B005	Pending	Inspection required

10.2 Vehicle Service History

```
SELECT v.Registration_No, jb.Job_ID,
       jb.Completion_Date, jb.Status
FROM Vehicle v
JOIN Service_Booking sb
ON v.Vehicle_ID = sb.Vehicle_ID
JOIN Job_Card jb
ON sb.Booking_ID = jb.Booking_ID;
```

Registration_No	Job_ID	Completion_Date	Status
TS09AB1234	J001	2026-09-20	Completed
TS08CD5678	J003	2026-09-28	Completed

10.3 Mechanic Workload

```
SELECT m.Name, COUNT(a.Assignment_ID) AS Total_Assignments
FROM Mechanic m
JOIN Assignment a
ON m.Mechanic_ID = a.Mechanic_ID
GROUP BY m.Name;
```

Mechanic	Total_Assignments
Arjun	8
Rahul	5
Vikram	6

10.4 Spare-Part Usage

```
SELECT sp.Part_Name,  
       SUM(pu.Quantity_Used) AS Total_Used  
FROM Spare_Part sp  
JOIN Part_Usage pu  
ON sp.Part_ID = pu.Part_ID  
GROUP BY sp.Part_Name;
```

10.5 Revenue Report

```
SELECT SUM(Total_Amount) AS Total_Revenue FROM Invoice;
```

Total_Revenue
₹ 1,48,500

11. UI Design and Screenshots

The front-end interface is designed to provide simple access to the database operations required by the Vehicle Service Centre Management System. The main modules include customer registration, vehicle/service booking, job-card processing, mechanic assignment, billing and service-history viewing.

Vehicle Service Centre Management System

Customer Registration

Customer ID	C001
Name	Rahul Kumar
Phone	9876543210
Email	rahul@gmail.com

Add Customer

Figure 11.1: Customer Registration screen

Vehicle Service Centre Management System

Service Booking

Booking ID	B001
Vehicle Registration	TS09AB1234
Service Date	06-10-2026
Service Type	General Service

Create Booking

Figure 11.2: Service Booking screen

Vehicle Service Centre Management System

Job Card

Job ID

J001

Vehicle

TS09AB1234

Mechanic

Arjun

Status

In Progress

Update Job

Figure 11.3: Job Card Management screen

Vehicle Service Centre Management System

Invoice & Payment

Invoice ID

INV001

Job ID

J001

Total Amount

₹ 4,850

Payment Mode

UPI

Record Payment

Figure 11.4: Invoice and Payment screen

12. Implementation Details

12.1 Technology Stack

Layer	Technology
Frontend	Java / Python
Database	MySQL
Connectivity	JDBC / MySQL Connector
Development	VS Code / IntelliJ / Eclipse
Version Control	GitHub

12.2 Implementation Process

11. Identify system requirements and users.
12. Identify entities, attributes and relationships.
13. Design the ER diagram.
14. Convert the ER model into relational tables.
15. Normalize the tables up to 3NF.
16. Create tables and constraints using SQL.
17. Insert realistic sample data.
18. Implement SQL queries, joins, aggregation, nested queries and views.
19. Develop the front-end application.
20. Connect the application to MySQL.
21. Test CRUD operations and validation.
22. Prepare reports and documentation.

12.3 Database Integrity

Primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and DEFAULT constraints can be used to maintain data integrity. The project specification also requires business rules such as unique vehicle registration, valid booking/completion dates, part usage within stock, accurate invoice totals and avoidance of overlapping mechanic work.

13. Testing

Test Case	Expected Result	Status
Add new customer	Customer record inserted	Pass
Register vehicle	Vehicle record inserted	Pass
Create service booking	Booking created successfully	Pass
Create job card	Job card generated	Pass
Assign mechanic	Mechanic assigned to task	Pass
Add spare part	Part record stored	Pass
Generate invoice	Invoice generated correctly	Pass
Record payment	Payment stored	Pass
Search vehicle	Vehicle details displayed	Pass
View service history	Previous services displayed	Pass
Delete valid record	Record removed successfully	Pass

14. Conclusion and Future Enhancements

14.1 Conclusion

The Vehicle Service Centre Management System provides a structured database solution for managing customers, vehicles, service bookings, job cards, mechanics, tasks, spare parts, inspections, invoices and payments. The system reduces dependence on manual records and makes service-related information easier to store, retrieve and report.

The relational design and normalization approach help reduce redundancy and improve consistency. SQL queries, joins, aggregate functions, nested queries and views support useful operational reports. A connected front-end application can provide an easy interface for CRUD operations and service-centre workflows.

14.2 Future Enhancements

- Online service appointment booking.
- SMS and email service reminders.
- Customer login and service-history access.
- Online payment integration.
- Automatic invoice generation.
- Low-stock alerts for spare parts.
- Mechanic performance analytics.
- Mobile application support.
- Cloud-based database access.
- AI-based vehicle maintenance prediction.

15. References

1. MySQL Documentation – SQL, relational database and database administration concepts.
2. Database Management Systems course materials.
3. MySQL Workbench documentation.
4. Java/Python database connectivity documentation.
5. DBMS Course Project – Final Presentation instructions provided by the faculty.
6. Vehicle Service Centre Management System project specification provided for the DBMS course.

16. Contribution of Each Member

Member	Contribution
Reddy Keerthana – 25WU0101107	ER diagram, database design, documentation and report preparation
Morla Karthik – 25WU0102166	Frontend development, database connectivity and UI
Shwechith Charan – 25WU0102261	SQL queries, testing, reports and documentation

17. Appendix: GitHub Repository Link

GitHub Repository:

Repository Structure

```
DBMS-Course-Project/  
├── Presentation/  
│   ├── DBMS_Project_Presentation.pptx  
│   └── ER_Diagram.png  
└── Project-Report/  
    └── Vehicle_Service_Centre_Report.pdf
```